U.S. GDP Curve Fitting/Interpolation Project

Conner Jaquess

COE 311K Spring 2026 – Midterm Project

Introduction

For this midterm project, I will be analyzing the United States growth of GDP. The dataset I will be using are selected quarters from the U.S. Bureau of Economic Analysis (BEA) (see figure 1.1), defined by the project's instruction sheet as "seasonally adjusted annual rate of real GDP growth, in percent."

Quarter	GDP Growth (%)	Quarter	GDP Growth (%)
2010 Q1	1.7	2017 Q1	1.3
2011 Q1	0.1	2018 Q1	2.5
2012 Q1	2.3	2019 Q1	3.1
2013 Q1	2.7	2020 Q1	−5.1
2014 Q1	1.7	2020 Q2	−28.1
2014 Q3	5.0	2020 Q3	33.8
2015 Q2	3.0	2021 Q1	6.3
2016 Q1	1.5	2022 Q1	−1.6
2016 Q3	3.5	2023 Q2	2.4
2016 Q4	1.8	2023 Q4	3.3

FIGURE 1.1 - SELECTED QUARTERS FOR DATASET

Using only this data, three separate methods will be used to either interpolate or approximate the quarters in between:

- Cubic spline will be used to create one interpolated piecewise function that continuously connects through each point.
- Using the Vandermonde matrix to find a least squares solution, a polynomial of degree four will approximate the data.
- A simple linear least squares solution will also be tested as an approximation.

After analyzing each method, I will decide on a superior choice for GDP analysis, between cubic spline and polynomial fitting.

Part A

The first method, a cubic spline, will interpolate the data into one piecewise function. Specifically, this spline will be a natural spline.

According to an authorless article from geeksforgeeks.org, natural spline is defined as a cubic spline where the second derivatives of both boundary points are equal to zero. In the context of this analysis, it makes more sense to use a natural spline because it provides an indication of the behavior of spline's function before and after its bounds. If the spline wasn't natural, it would have erratic behavior at the endpoints and unfairly disregard the fact that the economy exists before and after the scope of this study.

The cubic spline was constructed by creating a tridiagonal matrix based on the second derivates of each of the twenty "knots" (data point used to construct spline). Cell [3] in the Python Notebook document contains an algorithm that constructs a tridiagonal matrix, and using another that utilizes the Thomas algorithm, the system is solved to find M (the second derivative) at each knot. By using each one of the twenty second derivatives, I evaluate the spline to find the coefficients of each degree three polynomial.

Additionally, for this model to fit the definition of a spline, it must be smooth. Specifically, for each internal point:

- C^0 – The spline must pass through each data point.
    - This can be proven by observing through the residual plot (Figure 1.3) that each data point is passed through.
- C^1 – The first derivative of the spline must be continuous at each internal knot.
    - The function that creates the tridiagonal matrix ensures that this must be true.
- C^2 – The second derivate of the spline must be continuous at each internal knot
    - Like C^1, the tridiagonal matrix constructor imposes a condition that this must be true.

Finally, now that we know the algorithm is valid, we can interpolate the subset to create a full-range graph.

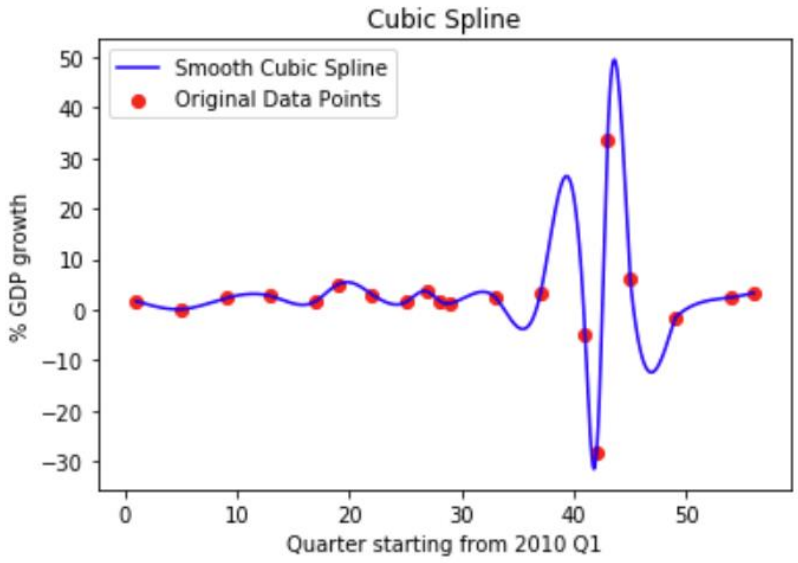

FIGURE 1.2 – CUBIC SPLINE

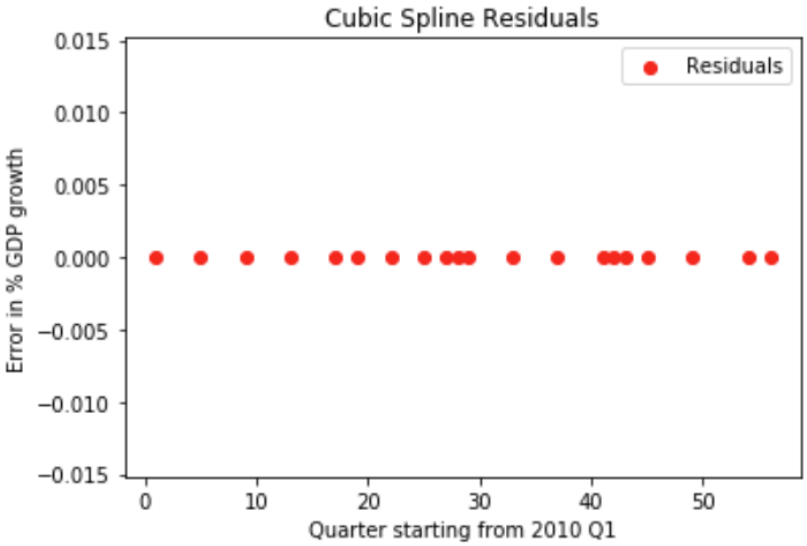

FIGURE 1.3 – CUBIC SPLINE RESIDUALS

Two things are immediately obvious. First, the interpolant is quite smooth and connected. After all, it had to satisfy the continuity conditions. However, the way it handles the COVID-19 spikes is problematic. Not only does it assume that the economy spikes massively before the pandemic, but it also assumes it suffers again after recovery; we know both of which are not true.

This presents the question of whether Runge's phenomenon is present. While the forced smoothness creates some inaccuracies for outliers, the Runge phenomenon does not occur here. If it were, we would see wild oscillations at the edges. Additionally, it can't really happen because cubic splines are of a low degree. Rather, the continuity conditions (required smoothness) are a trade-off for accuracy with outliers. In a spline, stability and exact interpolation cannot coexist.

Lastly, a smoothing spline would be a good implementation. Weighted least squares would also be an improvement but is less preferable simply because GDP growth is very noisy and a piecewise function inherently handles noise much better than a single polynomial. Both will still lower the chaos that the COVID-19 outliers cause.

Part B

For the next section, a polynomial fit and linear fit will be created using least squares.

Polynomial Fit

A Vandermonde matrix for a fourth-degree polynomial is created and is used to find the best-fitting coefficients for the data using solve_coefficients. Using those coefficients, the polynomial fit is plotted alongside the cubic spline:

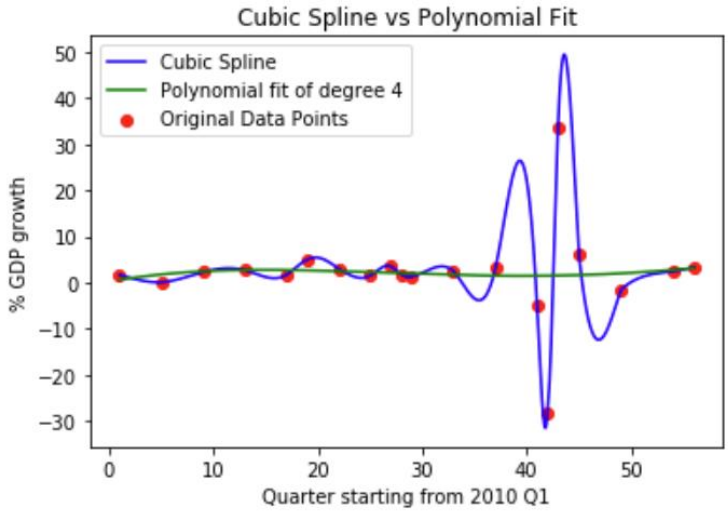

FIGURE 2.1 – CUBIC SPLINE VS POLYNOMIAL FIT

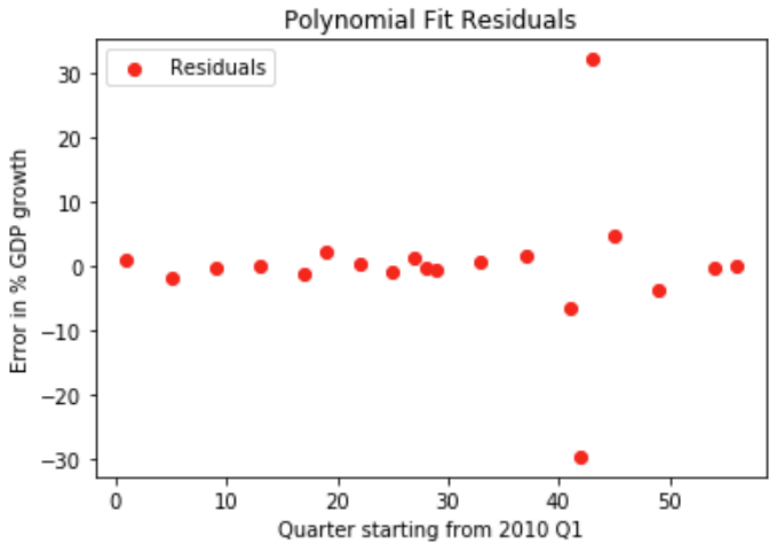

FIGURE 2.2 – POLYNOMIAL FIT RESIDUALS

It's evident that this fit becomes an approximation rather than an interpolation, for the COVID-19 outliers are barely addressed. Also, a huge amount of noise is reduced by this fit. The only thing that this model (mostly) addresses properly is the direction, positive or negative, of the economic growth. Even with that considered, the fact that the spline interpolates (goes through every point) while the polynomial fit does not is sufficient to justify that the spline shows the overall trend much more effectively.

Linear Least Squares

Using a simple stacked matrix, the NumPy least squares method is used to find the best linear representation of the data. Because the data is linear, we must remove the COVID-19 outliers from the data. A line simply cannot account for something like an outlier.

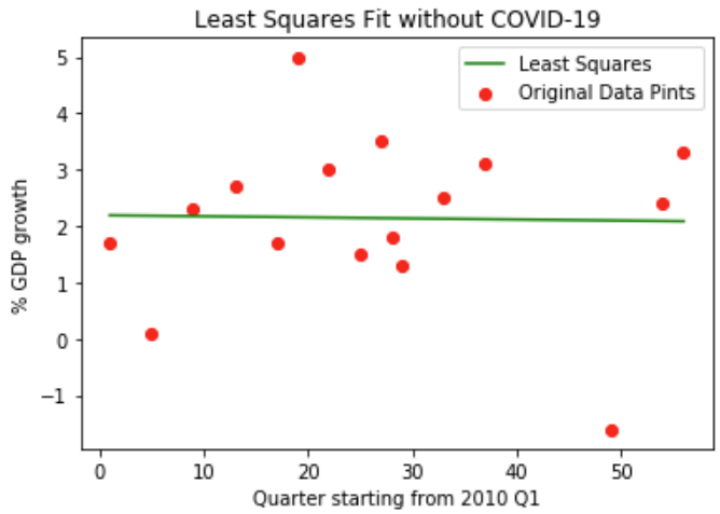

FIGURE 2.3 – LINEAR FIT

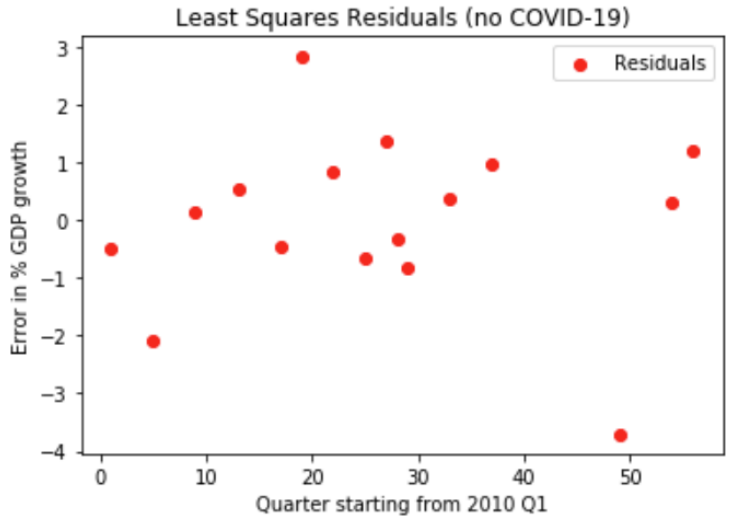

FIGURE 2.4 – LINEAR RESIDUAL PLOT

To be a bit informal, the plot is kinda sorta laughable. It takes in noisy data and makes a very calm model. It doesn't say much about the data. Regardless, the slope is -0.0019% and the offset in 2.1943%. These do uphold general economic trends; the slope is overall relatively constant at some positive growth. Although it's noisy, GDP growth will typically be positive, most of the time. In conclusion, the linear trend makes sense in some general ways, but for an analysis in more specific periods, it is ineffective.

Comparison

Based on the analysis above and the data in Figure 2.5, it is concluded that the polynomial fit is better than the linear fit.

	RMSE	R^2	Condition
Cubic Spline	0	1	7.93
Polynomial Fit	10.0314	0.0035	1.82e7
Polynomial Fit (no COVID)	1.1165	0.4017	1.66e7
Linear Fit (no COVID)	1.4432	0.0004	60.05

FIGURE 2.5 – MSE AND R^2

Of course, cubic spline will be "perfect" because it's an interpolant rather than an approximation. But for the comparison between polynomial fit and linear fit: if we make it a fair comparison by removing outliers for both, the polynomial fit is a much more accurate approximation because of its lower RMSE (measurement of general error).

Examining the condition numbers of each method's associated matrix, we can see that the polynomial fit was significantly more unstable than the others. This is because of how sensitive degree four polynomials are to change in larger data; the difference between 53^4 and 54^4 is huge, for example. For this reason, unless a different approach was taken, polynomial approximations could be very unstable with larger datasets or higher degrees. Cubic splines and linear fits, on the other hand, will pretty much always be a stable option.

What we can see through the levels of complexity of each method, though, is that while the cubic spline may be the most visually accurate, it's much harder to analyze numerically. A set of many polynomials is hard to take apart and compute, but the linear y = mx + b form makes it much easier to make simple predictions. The trade-off for easy interpretation is less accurate extrapolation. All models, of course, will be visually smooth, though!

Part C

If a policymaker had to choose between cubic spline or polynomial fitting to estimate GDP growth between two known quarters, a cubic spline would be the superior choice. The justification can be seen below.

- Smoothness
    - Both models are inherently smooth. A cubic spline has continuity conditions, and a polynomial fit is a singular model. While both models are smooth, a spline has more bends and curves, which allows additional versatility.
- Accuracy
    - In the above models, the cubic spline is obviously more accurate because it passes through every point. If a higher degree polynomial was used, there could potentially be higher accuracy at the data points, but the Runge's phenomenon could come into play quite badly. For that reason, the cubic spline is the better choice for accuracy.
- Sensitivity to Outliers
    - The cubic spline addresses the outliers. While it may not be perfect, the polynomial hardly acknowledges COVID-19 at all. Once again, cubic spline is a better option.
- Big O Analysis
    - Big O is an analytical representation of how long it takes to do a process based on n data.
    - Cubic spline has a Big O of O(n), meaning the amount of time it takes to compute is linearly proportional to the amount of data. This is because the Thomas Algorithm is used.
    - Polynomial fit has a Big O of O(n*d^2) with d being the degree of the polynomial. This slow element comes from Vandermonde matrix multiplication.
    - The linear fit has a Big O of O(n). Its computation is more straightforward than the cubic spline, so it's slightly faster. It is not within this comparison, though.
    - The cubic spline is faster to compute than a polynomial, especially as the polynomial's degree increases.

In conclusion, the cubic spline is the best option for interpolating data between two known quarters. It is superior in terms of smoothness, accuracy, sensitivity to outliers, and computation time.